

# SLAM & Navigation

## with TurtleBot3

ROS1 Noetic · Ubuntu 20.04 · CSE4020 Capstone Design

---

**Dr. Oybek Eraliev**

Department of Computer Science & Engineering



**TurtleBot3**  
**Burger / Waffle Pi**

## AGENDA

# Lecture Overview

*What we'll cover today*

01

## Introduction & Background

Why SLAM? TurtleBot3 hardware & sensors

03

## Running SLAM

Launch files, RViz, saving maps

05

## Navigation Stack

Costmaps, planners, move\_base

02

## SLAM Fundamentals

Pipeline, OGM, GMapping algorithm

04

## Localization (AMCL)

Particle filter, initial pose, parameters

06

## Sending Goals & Monitoring

RViz goals, Python action client

# Why SLAM & Navigation?

*The path to robot autonomy*



*Autonomous robots need all four capabilities working together*

Without a map, a robot cannot plan efficient paths

SLAM solves both mapping and localization simultaneously

Without localization, a robot cannot know where it is on the map

Navigation uses the map + pose to move to any goal autonomously

# TurtleBot3 Overview

*Hardware, sensors, and ROS1 ecosystem*

## TurtleBot3 Burger

- 2-wheel differential drive
- Single-layer platform
- Raspberry Pi 3B+
- OpenCR 1.0 (ARM Cortex-M7)
- 360° LiDAR — LDS-02
- IMU (MPU9250)
- ~138mm × 178mm × 192mm

## TurtleBot3 Waffle Pi

- 2-wheel differential drive
- Multi-layer platform
- Raspberry Pi 3B+
- OpenCR 1.0 (ARM Cortex-M7)
- 360° LiDAR — LDS-01 or HLS-LFCD2
- IMU + Camera (Pi Camera)
- ~281mm × 306mm × 141mm



**Key sensor for SLAM: LDS-01 LiDAR — publishes /scan at 5 Hz, 360° coverage, 0.12m–3.5m range**

# What is SLAM?

*Simultaneous Localization And Mapping*

S

Simultaneous

L

Localization

A

And

M

Mapping

## The Chicken-and-Egg Problem

To build a map, a robot needs to know its location.

To localize, a robot needs a map.

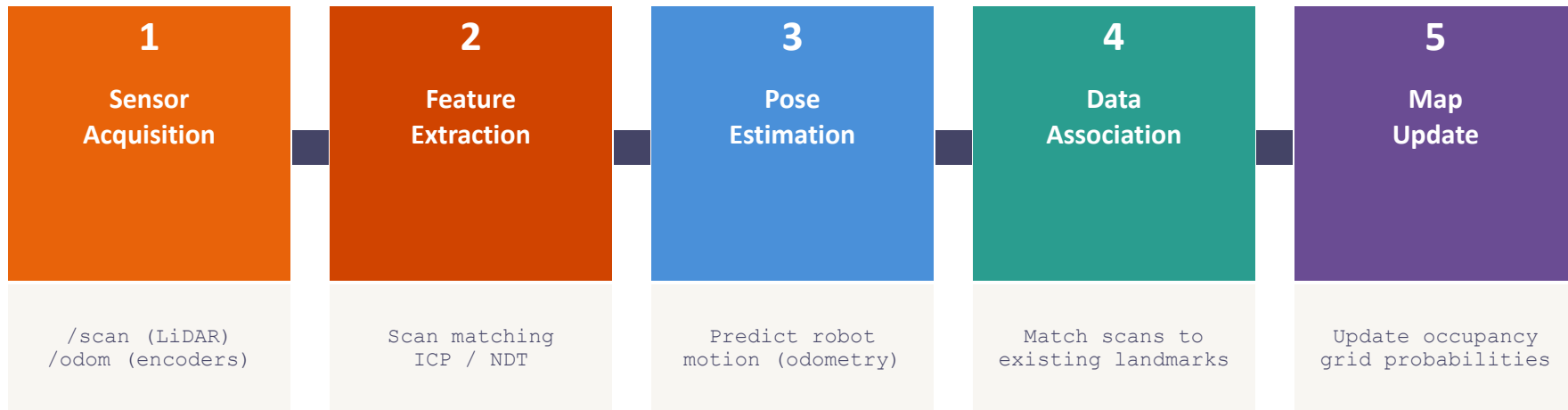
SLAM solves both problems at the same time!

```
Given sensor data  $z_{1:t}$   
and controls  $u_{1:t}$   
Estimate: map  $m$  and pose  $x_t$ 
```

*Used in: autonomous driving, warehouse robots, drones, vacuum cleaners, surgical robots*

# SLAM Pipeline

*From raw sensor data to a consistent map*



 This loop runs continuously as the robot moves — each new scan refines both the map and the robot's estimated pose

# Types of SLAM Algorithms

*Filter-based vs. Graph-based approaches*

## Filter-Based SLAM

- EKF-SLAM: Extended Kalman Filter — linearizes nonlinear motion
- Particle Filter (FastSLAM): samples possible robot poses
- GMapping: Rao-Blackwellized particle filter ← TurtleBot3 default
- Incremental: processes each sensor reading as it arrives
- Good for small-to-medium environments

## Graph-Based SLAM

- Poses = nodes in a graph; sensor constraints = edges
- Loop closure: detects revisited places to correct drift
- Hector SLAM: no odometry needed (scan matching only)
- Cartographer (Google): 2D/3D graph SLAM — TurtleBot3 optional
- Better for large environments with loop closures

★ TurtleBot3 uses GMapping (particle filter) by default in ROS1 — we'll focus on this

# LiDAR-Based SLAM

*How 2D laser scan data drives map building*

## How LDS-01 LiDAR Works

Rotates 360° emitting infrared pulses  
Measures distance to obstacles per angle  
Publishes sensor\_msgs/LaserScan on /scan  
Angular resolution:  $\sim 1^\circ$  | 5 Hz scan rate

## LaserScan Message Key Fields

angle\_min / angle\_max: scan range (rad)  
angle\_increment: step per measurement  
ranges[]: array of distance values (meters)  
range\_min / range\_max: valid distance limits

## ROS Topic Check

```
# View LiDAR data in terminal
rostopic echo /scan | head -30

# Check publish rate
rostopic hz /scan

# Visualize in RViz → Add → LaserScan
# Topic: /scan
# Style: Points | Size: 0.03
```

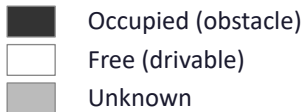
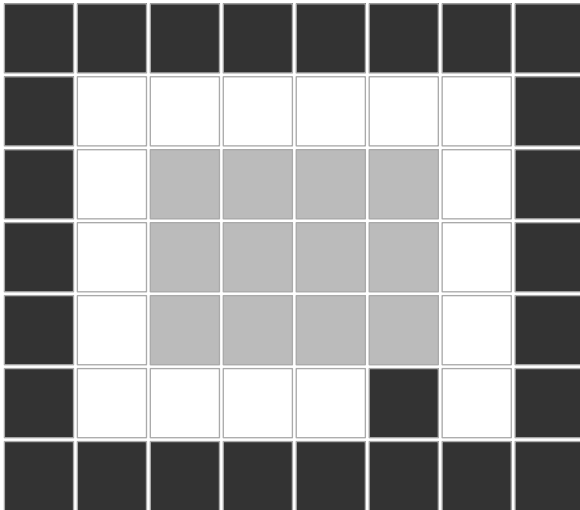
## Scan Matching

Two consecutive scans are aligned to estimate how much the robot moved. Combined with odometry, this produces a pose estimate for the particle filter.



# Occupancy Grid Maps

*The standard map format in ROS1*



## Grid Structure

2D array of cells (e.g. 384×384)  
Each cell = fixed real-world size (default 0.05m)  
Cell value: 0 (free), 100 (occupied), -1 (unknown)

## nav\_msgs/OccupancyGrid

Published on: /map  
Resolution: 0.05 m/cell (5 cm)  
Origin: reference frame in the world  
Header: frame\_id = 'map'

## Saving the Map

map\_server saves two files:  
.pgm — pixel image (white/black/gray)  
.yaml — metadata (resolution, origin, thresholds)

# GMapping Algorithm

*Rao-Blackwellized Particle Filter SLAM*

**Core Idea: Each particle carries a complete pose hypothesis AND its own map**

## Predict

Sample new poses from motion model (odometry).  
Each particle moves according to  $u_t$  with noise.

## Update

Weight each particle by how well its laser scan  $z_t$  matches its own map. Higher weight = better match.

## Resample

Draw  $N$  new particles proportional to weights.  
Particles with low weight are discarded.

## Map Update

Each surviving particle updates its occupancy grid using the current scan at its estimated pose.

Key parameters in `slam_gmapping`: `particles` (default 30) · `maxUrange` · `delta` (cell size) · `linearUpdate` · `angularUpdate`

# SLAM Packages in ROS1

*Choosing the right algorithm for TurtleBot3*

**slam\_gmapping**

★ DEFAULT

✓ Easy to use, well-tuned for TurtleBot3, built-in ROS1 support

✗ Requires odometry, may drift in large environments

```
roslaunch turtlebot3_slam
turtlebot3_slam.launch
slam_methods:=gmapping
```

**hector\_slam**

NO ODOM

✓ No odometry needed — scan matching only, great for flying robots

✗ Sensitive to featureless environments, no loop closure

```
roslaunch turtlebot3_slam
turtlebot3_slam.launch
slam_methods:=hector
```

**cartographer**

ADVANCED

✓ Graph SLAM with loop closure, accurate for large maps

✗ Higher CPU usage, more complex configuration

```
roslaunch turtlebot3_slam
turtlebot3_slam.launch
slam_methods:=cartographer
```

# TurtleBot3 SLAM Setup

*Environment configuration before launching*

## Step 1 — Install TurtleBot3 packages (if not already installed)

```
sudo apt install ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations
```

## Step 2 — Set TurtleBot3 model environment variable

```
# Add to ~/.bashrc for persistence
export TURTLEBOT3_MODEL=burger # or waffle_pi
source ~/.bashrc
```

## Step 3 — Launch Gazebo simulation (for simulation-based work)

```
# Terminal 1: Launch simulation world
roslaunch turtlebot3_gazebo turtlebot3_world.launch
```



On real TurtleBot3: ssh into the robot, run bringup first: `roslaunch turtlebot3_bringup turtlebot3_robot.launch`

# Launching SLAM

*Running GMapping with TurtleBot3*

**You need 3 terminals running simultaneously:**

## Terminal 1 — Gazebo Simulation

```
roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

## Terminal 2 — SLAM (GMapping)

```
export TURTLEBOT3_MODEL=burger  
roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping
```

## Terminal 3 — Teleoperation (to drive and build map)

```
export TURTLEBOT3_MODEL=burger  
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

Drive slowly and cover all areas of the environment for a complete map. Re-visit areas to improve accuracy.

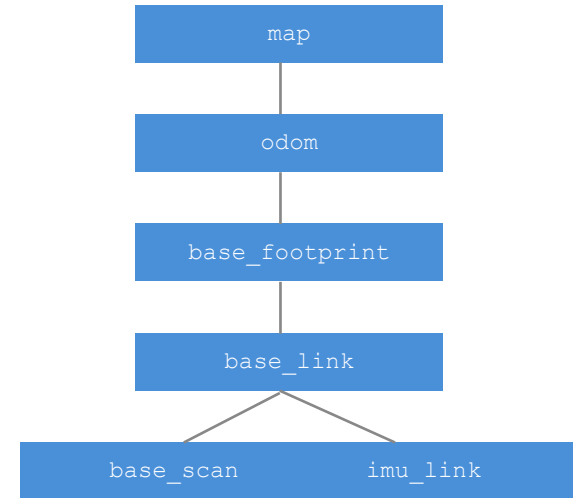
# Visualizing the Map in RViz

*Key topics and display settings during SLAM*

## Essential RViz Display Panels

| Display Type | Topic / Setting | Purpose                    |
|--------------|-----------------|----------------------------|
| Map          | /map            | Occupancy grid being built |
| LaserScan    | /scan           | Live LiDAR readings        |
| RobotModel   | —               | 3D model of TurtleBot3     |
| TF           | —               | Coordinate frame tree      |
| Odometry     | /odom           | Wheel encoder pose         |
| PoseArray    | /particlecloud  | AMCL particle cloud        |

## TF Frame Hierarchy (during SLAM)



# Saving the Map

*Using map\_saver to persist your occupancy grid*

Once satisfied with the map, save it before closing SLAM:

```
# Terminal 4 — Save map (while SLAM is still running!)
roslaunch map_server map_saver -f ~/maps/my_map

# This creates two files:
#   ~/maps/my_map.pgm   ← grayscale image of the map
#   ~/maps/my_map.yaml  ← metadata file
```

The .yaml metadata file contains:

```
image: my_map.pgm
resolution: 0.050000    # meters per pixel
origin: [-10.0, -10.0, 0] # lower-left corner (x,
y, yaw)
negate: 0
occupied_thresh: 0.65   # p > 0.65 → occupied
free_thresh: 0.196      # p < 0.196 → free
```

PGM Pixel Interpretation

| Pixel Value | Meaning  |
|-------------|----------|
| 0 (black)   | Occupied |
| 254 (white) | Free     |



**Never close SLAM before saving the map! The in-memory map is lost once the node shuts down.**

# What is Localization?

*Answering: 'Where am I on this map?'*

## SLAM vs. Localization — When to use which?

|             | SLAM                     | Localization (AMCL)         |
|-------------|--------------------------|-----------------------------|
| Map         | Builds the map           | Uses a saved map            |
| Pose        | Estimates simultaneously | Estimates against known map |
| When to use | First-time exploration   | Repeated missions           |
| ROS Package | slam_gmapping            | amcl                        |
| Map Topic   | Publishes /map           | Subscribes to /map          |

Typical workflow:

① Run SLAM once → Save map → ② Every future session: load map + run AMCL



# AMCL — Adaptive Monte Carlo Localization

*Particle filter localization against a known map*

|   |                       |                                                                                     |
|---|-----------------------|-------------------------------------------------------------------------------------|
| 1 | <b>Initialize:</b>    | Spread N particles uniformly across the map (or around an initial pose hint)        |
| 2 | <b>Motion Update:</b> | Move particles according to odometry (with noise) — robot moved, so shift particles |
| 3 | <b>Sensor Update:</b> | Weight each particle: compare expected laser scan from that pose vs actual /scan    |
| 4 | <b>Resample:</b>      | Draw new particle set weighted by likelihood. Particles converge to true pose       |
| 5 | <b>Adapt:</b>         | AMCL adds/removes particles dynamically — more when uncertain, fewer when confident |

## Key AMCL Parameters

|                                  |                               |
|----------------------------------|-------------------------------|
| <code>min_particles</code>       | Default: 500                  |
| <code>max_particles</code>       | Default: 2000                 |
| <code>update_min_d</code>        | Min translation (m) to update |
| <code>update_min_a</code>        | Min rotation (rad) to update  |
| <code>laser_max_beams</code>     | Beams used per scan           |
| <code>odom_model_type</code>     | diff / omni                   |
| <code>transform_tolerance</code> | TF publish latency (s)        |

# AMCL in ROS1

*Topics, services, and launch commands*

```
# Load saved map and run AMCL
roslaunch turtlebot3_navigation turtlebot3_navigation.launch \
  map_file:=$HOME/maps/my_map.yaml
```

## Key Topics & Services

| Topic / Service      | Direction  | Description                               |
|----------------------|------------|-------------------------------------------|
| /scan                | Subscribes | LiDAR input for particle weighting        |
| /tf                  | Subscribes | Odometry transforms (odom→base_footprint) |
| /map                 | Subscribes | Saved occupancy grid                      |
| /amcl_pose           | Publishes  | Best pose estimate with covariance        |
| /particlecloud       | Publishes  | All particle poses (visualize in RViz)    |
| ~global_localization | Service    | Randomize all particles (if lost)         |

## Lost Robot? Reset Localization

```
# Call service to scatter particles
rosservice call /global_localization

# Then drive robot around slowly
# Particles will re-converge
```

💡 After `global_localization`, drive the robot in small circles or along known features. The particle cloud will converge within seconds.

# Initial Pose Estimation

*Telling AMCL where the robot starts*

## Method 1 — RViz '2D Pose Estimate' Tool

- ① Open RViz (auto-launched by navigation.launch)
- ② Click '2D Pose Estimate' button in toolbar
- ③ Click on map at robot's approximate location
- ④ Drag arrow in direction robot is facing
- ⑤ Release — particles snap to that region
- ⑥ Drive robot slowly to refine convergence

### Why covariance matters:

Large covariance → particles spread wide  
(robot is uncertain about start position)

Small covariance → particles close together  
(robot knows exactly where it starts)

## Method 2 — Programmatic (Python)

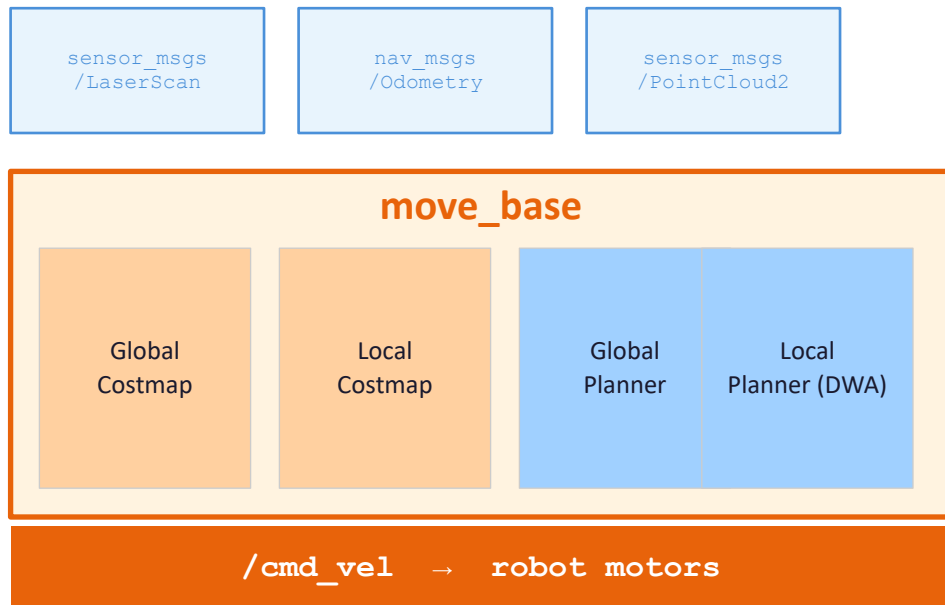
```
import rospy
from geometry_msgs.msg import
PoseWithCovarianceStamped

pub = rospy.Publisher('/initialpose',
    PoseWithCovarianceStamped, queue_size=1)
rospy.init_node('set_init_pose')
rospy.sleep(1.0)

msg = PoseWithCovarianceStamped()
msg.header.frame_id = 'map'
msg.pose.pose.position.x = 0.0
msg.pose.pose.position.y = 0.0
msg.pose.pose.orientation.w = 1.0
# Set covariance diagonal (xx, yy, th)
msg.pose.covariance[0] = 0.25
msg.pose.covariance[7] = 0.25
msg.pose.covariance[35] = 0.068
pub.publish(msg)
```

# ROS Navigation Stack Overview

*The complete autonomous navigation architecture*



## Inputs to move\_base

`/map` (occupancy grid)  
`/scan` (live LiDAR)  
`/odom` (wheel odometry)  
`/tf` (coordinate frames)  
`/move_base/goal` (target pose)

## What move\_base does

1. Creates global costmap from `/map`
2. Plans global path with A\* / Dijkstra
3. Creates local costmap from `/scan`
4. DWA generates velocity commands
5. Publishes `/cmd_vel` to drive robot

# Costmaps

*How the robot represents safe and dangerous space*

## Global Costmap

- Built from the saved static map (/map)
- Used by the global planner for full path planning
- Updated rarely (mostly static)
- Covers the entire known environment
- Resolution typically matches map (0.05m)

## Local Costmap

- Built from live sensor data (/scan)
- Used by the local planner (DWA) for real-time avoidance
- Rolls with the robot — limited window (e.g.,  $3\text{m} \times 3\text{m}$ )
- Handles dynamic obstacles (people, moving objects)
- Updated every control cycle (~5-10 Hz)

## Inflation Radius

Obstacles in the costmap are 'inflated' outward by a buffer zone (e.g., 0.1m). This ensures the robot body stays clear of walls. Set in `costmap_common_params.yaml`: `inflation_radius: 0.1` and `robot_radius: 0.105` (Burger).

# Global Planner

*Finding the optimal path from start to goal*

## What the Global Planner Does

Receives a goal pose (x, y,  $\theta$ ) on the global costmap  
 Plans the shortest collision-free path using Dijkstra or A\*  
 Path is a sequence of waypoints in the /map frame  
 Published to /move\_base/NavfnROS/plan (visualize in RViz)

## Default: NavFn (Dijkstra)

Wavefront expansion from goal backwards to start  
 Guarantees optimal path in known static costmap  
 Alternative: Global Planner plugin using A\* (faster)  
 Switch with: base\_global\_planner: navfn/NavfnROS

## A\* vs Dijkstra — Quick Comparison

|                | Dijkstra   | A*                        |
|----------------|------------|---------------------------|
| Heuristic      | None       | Distance to goal          |
| Optimality     | Guaranteed | Guaranteed (admissible h) |
| Speed          | Slower     | Faster                    |
| Default in ROS | ✓ NavFn    | Optional plugin           |

```
# In move_base.launch or param
file:
<param name="base_global_planner"
  value="navfn/NavfnROS"/>
# Or A*:
<param name="base_global_planner"
  value="global_planner/
GlobalPlanner"/>
<param name="GlobalPlanner/
use_dijkstra"
  value="false"/>
```

# Local Planner — DWA

*Dynamic Window Approach for real-time obstacle avoidance*

DWA samples  $(v, \omega)$  pairs within a 'dynamic window' of physically reachable velocities, simulates each trajectory, and picks the one that best balances goal progress, obstacle clearance, and speed.

1

## Sample

Generate candidate (linear\_vel, angular\_vel) pairs limited by robot's acceleration constraints

2

## Simulate

Forward-simulate each  $(v, \omega)$  for a short time horizon (~0.5–1.5s) to get a trajectory arc

3

## Score

Score each trajectory: heading to goal + clearance from obstacles + velocity (speed reward)

4

## Select

Pick the highest-scoring feasible trajectory and publish its  $(v, \omega)$  as `/cmd_vel`

Key params in `dwa_local_planner_params.yaml`: `max_vel_x` · `min_vel_x` · `max_vel_theta` · `acc_lim_x` · `acc_lim_theta` · `xy_goal_tolerance` · `yaw_goal_tolerance`

# move\_base Node

*The central coordinator of the Navigation Stack*

## move\_base Action Interface

```
# move_base uses ROS actionlib (MoveBase action)
# Action name: /move_base

# Goal:      geometry_msgs/PoseStamped  (target x,y,θ)
# Result:    empty — success/failure via status
# Feedback:  geometry_msgs/PoseStamped  (current base pos)

# Topics auto-created by actionlib:
# /move_base/goal
# /move_base/cancel
# /move_base/result
# /move_base/feedback
# /move_base/status
```

## Recovery Behaviors

If the robot gets stuck: (1) Clear costmaps → (2) Rotate in place → (3) Back up → (4) Abort goal

## Useful Commands & Debug

```
# Check move_base status
rostopic echo /move_base/status
```

```
# View global plan in RViz:
# Add → Path → /move_base/NavfnROS/
plan
```

```
# Clear costmaps manually:
rosservice call /move_base/
clear_costmaps
```

```
# Cancel current goal:
rostopic pub /move_base/cancel
actionlib_msgs/GoalID -- {}
```



# Navigation Configuration Files

*YAML parameter files for TurtleBot3*

## costmap\_common\_params\_burger.yaml

```
obstacle_range: 3.0
raytrace_range: 3.5
footprint: [[-0.105, -0.105], [-0.105, 0.105],
            [0.041, 0.105], [0.041, -0.105]]
inflation_radius: 0.1
map_type: costmap
```

## global\_costmap\_params.yaml

```
global_costmap:
  global_frame: map
  robot_base_frame: base_footprint
  update_frequency: 10.0
  publish_frequency: 10.0
  static_map: true
```

## local\_costmap\_params.yaml

```
local_costmap:
  global_frame: odom
  robot_base_frame: base_footprint
  update_frequency: 10.0
  publish_frequency: 10.0
  static_map: false
  rolling_window: true
  width: 3    height: 3    resolution: 0.05
```

## dwa\_local\_planner\_params\_burger.yaml

```
DWAPlannerROS:
  max_vel_x: 0.22  min_vel_x: -0.22
  max_vel_theta: 2.75
  acc_lim_x: 2.5    acc_lim_theta: 3.2
  xy_goal_tolerance: 0.05
  yaw_goal_tolerance: 0.17
```

# Launching Navigation

*Full startup sequence for autonomous navigation*

**Complete startup — 3 terminals required:**

## Terminal 1 — Gazebo Simulation

```
roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

## Terminal 2 — Navigation Stack (loads map + AMCL + move\_base)

```
export TURTLEBOT3_MODEL=burger  
roslaunch turtlebot3_navigation turtlebot3_navigation.launch \  
  map_file:=$HOME/maps/my_map.yaml
```

## Terminal 3 — (Optional) RViz standalone view

```
roslaunch turtlebot3_navigation turtlebot3_navigation.launch  
# RViz auto-launches — or open separately:  
rviz -d $(rospack find turtlebot3_navigation)/rviz/turtlebot3_nav.rviz
```

After launch: set initial pose in RViz using '2D Pose Estimate', then send goals using '2D Nav Goal'

# Sending Goals via RViz

*Using the '2D Nav Goal' tool interactively*

1

## Open RViz

Navigation stack auto-launches RViz with the TurtleBot3 navigation config loaded.

3

## Wait for AMCL

Observe `/particlecloud` in RViz converging around the robot's true location.

5

## Watch the Robot

Global planner draws a path. Local planner (DWA) executes it while avoiding obstacles.

2

## Set Initial Pose

Click '2D Pose Estimate' → click on map → drag arrow to set robot's starting orientation.

4

## Send Nav Goal

Click '2D Nav Goal' → click destination on map → drag to set desired final orientation.

6

## Goal Reached

Robot stops at goal pose. `move_base` publishes SUCCEEDED status on `/move_base/status`.

RViz toolbar shortcut: 'G' key = 2D Nav Goal | 'P' key = 2D Pose Estimate

# Sending Goals Programmatically

*Python action client for move\_base*

## move\_base Action Client — Python

```
#!/usr/bin/env python3
import rospy, actionlib, math
from move_base_msgs.msg import MoveBaseAction, MoveBaseGoal

def send_goal(x, y, yaw=0.0):
    client = actionlib.SimpleActionClient('move_base', MoveBaseAction)
    client.wait_for_server()           # block until server ready
    goal = MoveBaseGoal()
    goal.target_pose.header.frame_id = "map" # always use 'map' frame
    goal.target_pose.header.stamp = rospy.Time.now()
    goal.target_pose.pose.position.x = x
    goal.target_pose.pose.position.y = y
    goal.target_pose.pose.orientation.z = math.sin(yaw / 2.0)
    goal.target_pose.pose.orientation.w = math.cos(yaw / 2.0)
    client.send_goal(goal)
    client.wait_for_result()           # block until done
    return client.get_state()          # SUCCEEDED=3, ABORTED=4

if __name__ == '__main__':
    rospy.init_node('nav_client')
    print(send_goal(1.5, 0.8, 0.0))    # go to (1.5m, 0.8m)
```

# Monitoring Navigation

*Debugging and visualizing in real-time*

## RViz Displays to Enable

- Map: /map — static occupancy grid
- LaserScan: /scan — live sensor readings
- Path: /move\_base/NavfnROS/plan — global path
- Path: /move\_base/DWAPlannerROS/local\_plan — local trajectory
- PoseArray: /particlecloud — AMCL particles
- Pose: /amcl\_pose — best estimate pose

## Useful Command-Line Monitors

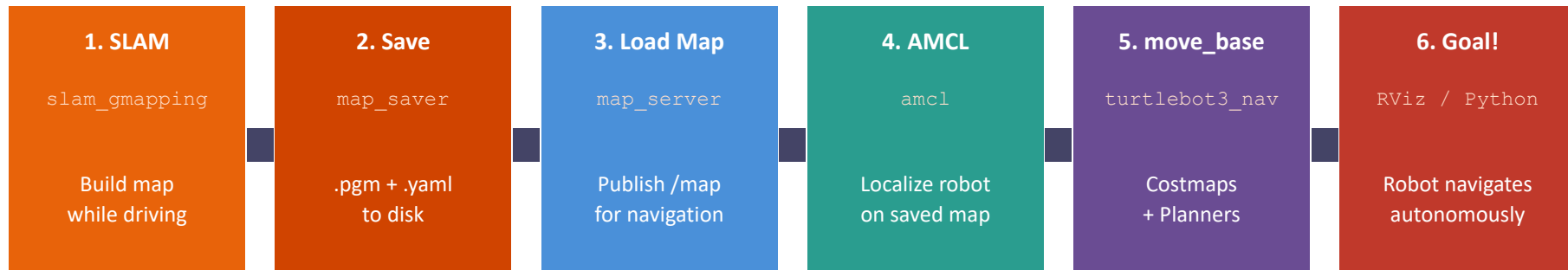
- rostopic echo /move\_base/status — goal status
- rostopic echo /amcl\_pose — current pose estimate
- rostopic echo /cmd\_vel — velocity commands
- rosrn rqt\_graph rqt\_graph — node topology
- rosrn rqt\_plot rqt\_plot /cmd\_vel/linear/x — live plot
- rostopic echo /move\_base/result — success/failure



rqt\_bag is useful for recording a full navigation session and replaying it offline for debugging

# Full Pipeline Summary

*SLAM → Save Map → AMCL → Navigation → Goal*



## Quick-Start Commands Reference

```
# 1. Gazebo simulation
roslaunch turtlebot3_gazebo turtlebot3_world.launch
# 2. SLAM (GMapping)
roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping
# 3. Teleoperate to build map
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
# 4. Save map
roslaunch map_server map_saver -f ~/maps/my_map
# 5. Launch navigation (loads map + AMCL + move_base)
roslaunch turtlebot3_navigation turtlebot3_navigation.launch map_file:=$HOME/maps/my_map.yaml
```

# Key Takeaways

---

- SLAM builds a map and localizes simultaneously — the chicken-and-egg problem solved by GMapping
- The occupancy grid map (.pgm + .yaml) is saved once and reused in every subsequent session
- AMCL localizes the robot on the saved map using a particle filter — always set initial pose first
- The Navigation Stack (move\_base) combines global A\*/Dijkstra planning with local DWA avoidance
- Configuration YAML files tune costmap parameters and DWA velocities for TurtleBot3's dimensions
- Goals can be sent interactively via RViz or programmatically via the move\_base action client

## Q&A

Questions?